14: The Kubernetes API

Understanding the Kubernetes API, and how it works, is vital to mastery of Kubernetes. However, it can be extremely confusing if you're new to APIs and not comfortable with terms like *RESTful*. If that's you, this chapter will blow away the confusion and get you up-to-speed with the fundamentals of the Kubernetes API.

The chapter is divided as follows.

- Kubernetes API big picture
- The API server
- The API

A couple of quick things before we get cracking with the chapter…

First up. I've made no attempt to make this chapter jargon-free. In fact, I've intentionally included a lot of jargon so you get comfortable with it. I've also included an extended chapter summary full of jargon you should be able to understand if you've read the whole chapter.

Last up. I've included and bunch of hands-on commands and exercises. I've made them as simple as possible to follow along with, and I highly recommend you do. They'll re-inforce the theory you're about to learn.

OK, let's do this.

Kubernetes API big picture

Let's start out with the super high level…

Kubernetes is *API centric*. This means everything in Kubernetes is about the API, and everything goes through the API and API server. We'll get into detail in a second, but for now, let's just look at the big picture.

Clients send requests to Kubernetes to create, read, update, and delete objects such as Pods and Services. For the most part, you'll use `kubectl` to send these requests, however, you can craft them in code or use API testing and development tools to generate them. The point is, no matter how you generate requests, they go to the *API server* where they're authenticated and authorized. Assuming they pass the auth tests, they're executed on the cluster. If it's a create request, the object is deployed to the cluster and the serialized state of it is persisted to the cluster store.

The overall process is depicted in Figure 14.1 and shows the central nature of the API and API server. However, other components, such as the cluster store and scheduler, are also very important.

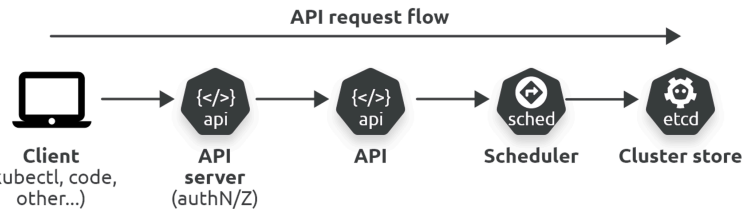

Figure 14.1

JSON serialization

We've already introduced a fair bit of jargon, so let's start busting some of it.

What is meant by "serialized state of an object".

Serialization is the process of converting an object into a string, or stream of bytes, so it can be sent over a network and persisted to a data store. The reverse process of converting a string or stream of bytes into and object is *deserialization*.

Kubernetes serializes objects, such as Pods and Services, as JSON strings to be sent over HTTP. The process happens in both directions, with clients like `kubectl` serializing objects when posting to the API server, and the API server serializing responses back to clients. In the case of Kubernetes, the serialised state of objects are also persisted to the cluster store which is usually based on the etcd database.

So, in Kubernetes, serialization is the process of converting an object to into a JSON string to be sent over an HTTP connection and persisted to the cluster store.

However, as well as JSON, Kubernetes also supports Protobuf as a serialization schema. This is faster, more efficient, and scales better than JSON. But it's not as user-friendly when it comes to introspection and troubleshooting. At the time of writing, Protobuf is mainly used for internal cluster traffic, whereas JSON is used when communicating with external clients.

One final thing on serialization. When clients send requests to the API server, they use the `Content-Type` header to list the serialization schemas they support. For example, a client that only supports JSON will specify `Content-Type: application/json` in the HTTP header of the request. Kubernetes will honour this with a serialized response in JSON.

You'll see this in some of the examples later.

API analogy

While we're being super high level, let's consider a quick analogy that might help you conceptualise the Kubernetes API.

Amazon sells lots of stuff. That *stuff* is stored in warehouses and listed online on the Amazon website. You use a browser to search the website and buy stuff. 3rd-parties even sell their own stuff through Amazon and you use the same browser and website. When you buy stuff through the website, it gets delivered to you and you can start using it. The Amazon website even lets you track your stuff while it's being prepared and delivered. Once it's delivered, it's yours and Amazon is only involved if you want to do things like order more or send stuff back.

Well, it's pretty much the same with the Kubernetes API.

Kubernetes offers lots of objects such as Pods, Services, and Ingresses. They're defined in the API and exposed through the API server. You use tools like `kubectl` to request them. 3rd-parties even define their own objects in Kubernetes and you use the same `kubectl` and API server to request them. When you request an object through the API server, it gets created on your cluster and you can start using it. The API server even lets you watch it while it's being created. Once it's created, Kubernetes continues to observe it and you can query its state through the API server. Actions like creating more and deleting objects are also done through the API server.

The comparison is shown in Figure 14.2, and a feature-for-feature comparison is shown in the following table. Keep in mind, however, it's just an analogy and not everything is a perfect match.

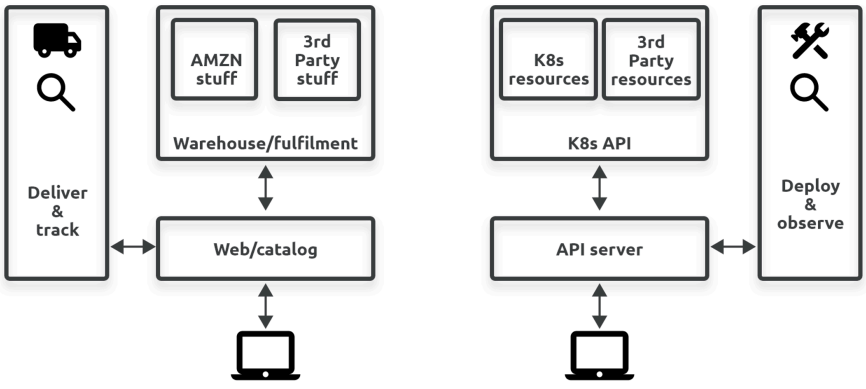

Figure 14.2

Amazon	Kubernetes
Stuff	Objects
Warehouse	API
Browser	kubectl
Amazon website	API server

To recap. All deployable objects, such as Pods, Services and Ingresses, are defined as resources in the API. If an object isn't defined in the API, you can't deploy it. This is the same with Amazon – you can only buy stuff that's listed on the website.

API Resources have properties you can view and configure. For example, Pod objects are defined in the API with all of the following properties (they have more properties than shown).

- metadata (name, labels, namespace, annotations...)
- restart policy
- service account name
- runtime class
- volumes

This is the same as buying things on Amazon. For example, when buying a USB cable, you can configure choices such as USB type, cable length, and even cable colour.

To deploy a Pod, you send a Pod YAML file to the API server. Assuming it's valid and you're authorized to create Pods, it gets deployed to the cluster. After that, you can query the API server to get its current status. When it's time to delete it, you send the delete request to the API server.

This is also the same as buying through Amazon. To buy the previously mentioned USB cable, you input all the colour, length and type options, and submit them to the Amazon website. Assuming it's in stock and you provide the funds, it gets shipped to you. After that, you can use the website to track the shipment. If you have to return the item or make a complaint, you do all of that through the Amazon website.

OK, that's enough with analogies. Let's take a closer look at the API server.

The API server

The API server exposes the API over a secure RESTful interface using HTTPS. It acts as the front-end to the API and is a bit like Grand Central for Kubernetes – everything talks to everything else via REST API calls to the API server. For example:

- All `kubectl` commands go to the API server (creating, retrieving, updating, deleting objects)
- All node Kubelets watch the API server for new tasks and report status to the API server
- All control plane services communicate via the API server (components don't talk directly each other)

Let's demystify a bit more jargon.

The *API server* is a Kubernetes control plane service. This usually means it runs as a set of Pods in the `kube-system` Namespace on the control plane nodes of your cluster. If you build and manage your own Kubernetes clusters, you need to make sure the control plane is highly-available and has enough performance to keep the API server up-and-running and responding quickly to requests. If you're using a *hosted Kubernetes* cluster, the way the API server is implemented, including performance and availability, will be hidden from you.

The main job of the API server is to make API available to clients inside and outside the cluster. It uses TLS to encrypt the client connection, and it leverages of a bunch of authentication and authorization mechanisms to ensure only valid requests are accepted and actioned. Requests from internal and external sources all have to pass through the same authentication and authorization.

The API is *RESTful*. This is jargon for a modern web API that accepts CRUD-style requests via standard HTTP methods. *CRUD-style operations* are simple *create, read, update, delete* operations, and the common *HTTP methods* include POST, GET, PUT, PATCH, and DELETE.

The following table shows how HTTP methods, CRUD operations, and kubectl commands match-up. If you've read the chapter on API security, you'll know we use the term *verb* to refer to CRUD operations.

HTTP method	K8s CRUD verb	kubectl example
POST	create	$ kubectl create -f <filename>
GET	get list, watch	$ kubectl get pods
PUT/PATCH	update	$ kubectl edit deployment <deployment-name>
DELETE	delete	$ kubectl delete ingress <ig-name>

As you can see, method names, CRUD verb names, and `kubectl` command names don't always match. For example, a `kubectl` **edit** command requires a user be authorized to use the **update** verb, and will send an HTTP **PATCH** request to the API server.

It's common for the API server to be exposed on port 443 or 6443, but it's possible to configure it to operate on whatever port you require. Running the following command shows the address and port your Kubernetes cluster is exposed on.

```
$ kubectl cluster-info
Kubernetes master is running at https://0.0.0.0:52935
CoreDNS is running at https://0.0.0.0:52935/api/v1/namespaces/../services/kube-dns:dns/proxy
Metrics-svr is running at https://0.0.0.0:52935/api/v1/../../../https:metrics-server:/proxy
```

In the example, the cluster is exposed on `https://0.0.0.0:52935`. This means any local address on port 52935.

A word on REST and RESTful

You'll hear the terms REST and RESTful a lot. REST is short for REpresentational State Transfer, and it's the de facto standard for communicating with web-based APIs. Systems, such as Kubernetes, that use REST are often referred to as *RESTful*.

REST requests comprise a *verb* and a *path* to a resource. Verbs relate to actions, and are the standard HTTP methods you saw in the previous table. Paths are a URI path to the resource in the API.

> **Disambiguation**: We often use the term *verb* to refer to CRUD operations as well as HTTP methods. CRUD operations include *create, read, update* and *delete*, whereas the HTTP methods are *GET, POST, PUT, PATCH* and *DELETE*. If it confusing, the term "verb" is used to refer to an *action*.

The following example shows a `kubectl` command and associated REST path that will list all Pods in the shield Namespace

```
$ kubectl get pods --namespace shield
```

```
GET /api/v1/namespaces/shield/pods
```

To visualise this, start a `kubectl proxy` and use `curl` to generate the request. The `kubectl proxy` command exposes the API on your `localhost` adapter and takes care of authentication. You can use a different port.

```
$ kubectl proxy --port 9000 &
[1] 14774
Starting to serve on 127.0.0.1:9000
```

With the proxy running, use a tool like `curl` to form a request to the API server.

```
$ curl http://localhost:9000/api/v1/namespaces/shield/pods
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "484812"
  },
  "items": []
}
```

The example returned an empty list because there are no Pods running in the shield Namespace. Try another request to list all Namespaces.

```
$ curl http://localhost:9000/api/v1/namespaces
```

Responses from the API server include common HTTP response codes, content type, and the actual payload. As you learned earlier in the chapter, Kubernetes uses JSON as its preferred content type. As a result, the previous `kubectl get` command will result in an HTTP 200 (OK) response code, content type will be `application/json` and the payload will be a serialized JSON list of all Pods in the shield Namespace.

Run one of the previous `curl` commands again, but add the -v flag to see the send and receive headers. The response has been trimmed to fit the book and draw your attention to the most important parts.

```
$ curl -v http://localhost:9000/api/v1/namespaces/shield/pods

> GET /api/v1/namespaces/shield/pods HTTP/1.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: application/json
< X-Kubernetes-Pf-Flowschema-Uid: 499d0001-d874-4b06-ba...c37f7
< X-Kubernetes-Pf-Prioritylevel-Uid: aeb490e6-1890-41ab...94e82
<
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "487845"
  },
  "items": []
}
```

Lines starting with a › are header data *sent* by `curl`, whereas lines starting with a ‹ are header data returned to `curl` by the API server.

The › lines show `curl` sending a GET request to the `/api/v1/namespaces/shield/pods` REST path and telling the API server it can accept any valid serialization schema. The lines starting with ‹ show the API server returning an HTTP response code and using JSON as the serialization schema. The `X-Kubernetes` lines are additional priority and fairness settings specific to Kubernetes.

A word on CRUD

CRUD is an acronym for the four basic functions web APIs use to manipulate and persist objects – create, read, update, delete. As previously mentioned, the Kubernetes API exposes and implements CRUD-style operations via the common HTTP methods.

Let's consider an example.

The following JSON is from the "ns.json" file in the `api` folder of the book's GitHub repo. It defines a new Namespace object called "shield"

```
{
  "kind": "Namespace",
  "apiVersion": "v1",
  "metadata": {
    "name": "shield",
    "labels": {
      "chapter": "api"
    }
  }
}
```

You can create it with the `kubectl apply -f ns.json` command. Behind the scenes, `kubectl` creates a request to the API server using the HTTP POST method. This is why you'll occasionally see some documentation refer to "POSTing" to the API server. The POST method creates a new object of the specified resource type. In this example, it'll create a new Namespace object called "shield".

The following is a simplified example of the request header. The body will be the contents of the JSON file.

Request header:

```
POST https://<api-server>/api/v1/namespaces
Content-Type: application/json
Accept: application/json
```

The `Content-Type` and `Accept` fields of the header tell the API server the content is being sent in JSON, and, the client expects a response in JSON.

If the request is successful, the response will include a standard HTTP response code, content type, and actual payload.

```
HTTP/1.1 200 (OK)
Content-Type: application/json
{
    ...
}
```

Talk is cheap though… try posting it to the API server with the following curl command. You'll need a kubectl proxy process exposing the API server on port 9000 (`kubectl proxy --port 9000 &`) and you'll need to run the command from the directory containing the `ns.json` file.

```
$ curl -X POST -H "Content-Type: application/json" \
  --data-binary @ns.json http://localhost:9000/api/v1/namespaces

<Snip>
```

The `-X POST` argument forces `curl` to use the HTTP POST method. The `-H "Content-Type..."` tells the API server the request contains serialized JSON. The `--data-binary @ns.json` specifies the manifest file, and the URI is the address the API server is exposed on by `kubectl proxy`.

You can verify the new Namespace was created with the `kubectl get namespaces` command.

Feel free to delete the newly created Namespace by specifying the DELETE HTTP method.

```
$ curl -X DELETE -H "Content-Type: application/json" http://localhost:9000/api/v1/namespaces/shield
{
  "kind": "Namespace",
  "apiVersion": "v1",
  "metadata": {
    "name": "shield",
    <Snip>
  },
  "spec": {
    "finalizers": [
      "kubernetes"
    ]
  },
  "status": {
    "phase": "Terminating"
  }
}
```

In summary, the API server exposes the API over a secure RESTful interface that lets you manipulate and query the state of objects on the cluster. It runs on the control plane, which needs to be highly available and have enough performance to service requests quickly.

The API

The API is where all Kubernetes resources are defined. It's large, modular, and RESTful.

When Kubernetes was originally created, the API was monolithic in design with all resources existing in a single global namespace. However, as Kubernetes grew, it became necessary to divide the API into smaller more manageable groups. Figure 14.3 shows a simplified view of what it currently looks like.

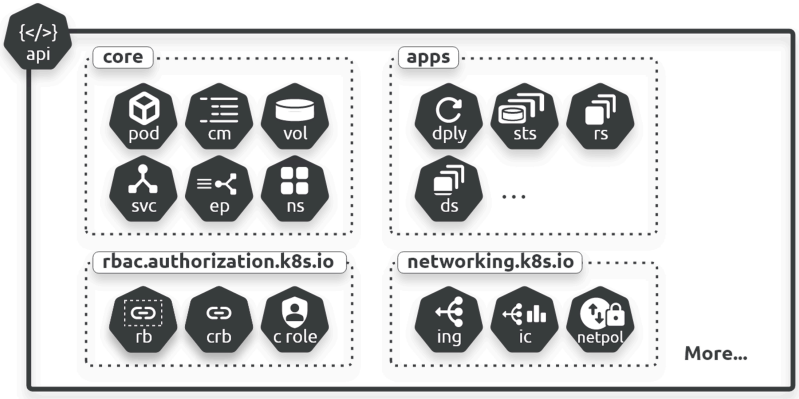

Figure 14.3

As you can see, the image shows the API with 4 groups. There are a lot more than 4, but the image only shows 4 for simplicity.

At the highest level, there are two types of API group.

- The **core** group
- The **named** groups

The core API group

Resources in the core group are mature objects that were created in the early days of Kubernetes before the API was divided into groups. They tend to be fundamental objects such as Pods, nodes, Services, Secrets, and ServiceAccounts. They're located in the API below `/api/v1`. The following table lists some example paths for resources in the core group. You'll sometimes see and hear these paths referred to as *REST paths*.

Resource	Path
Pods	/api/v1/namespaces/{namespace}/pods/
Services	/api/v1/namespaces/{namespace}/services/
Nodes	/api/v1/nodes/
Namespaces	/api/v1/namespaces/

Notice that some objects are namespaced and some aren't. Namespaced objects have longer REST paths as you have to include two additional segments – `.../namespaces/{namespace}/...` For example, listing all Pods in the "shield" Namespace requires the following path.

```
GET /api/v1/namespaces/shield/pods/
```

Expected HTTP response codes for read requests are either `200: OK` or `401: Unauthorized`.

On the topic of REST paths, GVR stands for group, version, resource, and can be a good way to remember the structure of REST paths in the Kubernetes API. A simple example is shown in Figure 14.4, but paths for namespaced objects include longer resource segments.

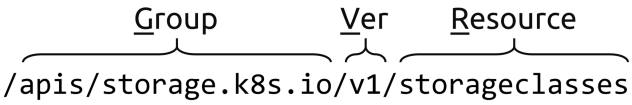

Figure 14.4

You shouldn't expect any new resources to be added to the core group. Anything new will be added to a *named group*.

Named API groups

The named API groups are the future of the API, and all new resources go into named groups. Sometimes we refer to them as "sub-groups"

Each of the named groups is a collection of related resources. For example, the "apps" group is where all resources that manage application workloads such as Deployments, ReplicaSets, DaemonSets, and StatefulSets are defined. Likewise, the "networking.k8s.io" group is where Ingresses, Ingress Classes, Network Policies, and other network-related resources exist. Notable exceptions to this pattern are older resources in the core group that came along before the named groups existed. For example, Pods and Services are both in the core group. However, if they were invented today, Pods would probably go in the "apps" group, and Services in the "networking.k8s.io" group.

Resources in the named groups live below the `/apis/{group-name}/{version}/` path. The following table lists some examples.

Resource	Path
Ingress	/apis/networking.k8s.io/v1/namespaces/{namespace}/ingresses/
RoleBinding	/apis/rbac.authorization.k8s.io/v1/namespaces/{namespace}/rolebindings/
ClusterRole	/apis/rbac.authorization.k8s.io/v1/clusterroles/
StorageClass	/apis/storage.k8s.io/v1/storageclasses/

Notice how the URI paths for named groups start with `/apis` and include the name of the group. This is different to the core group that starts with `/api` in the singular and doesn't include a group name. In fact, in places you'll see the core API group referred to by empty double quotes ("""). This is because when the API was first designed, no thought was given to groups – everything was "just in the API".

Dividing the API into smaller groups makes it more scalable and easier to navigate. It also makes it easier to extend.

The following commands are good for seeing API related info for your clusters.

`kubectl api-resources` is great for seeing which resources are available on your cluster, as well as which API groups they're served from. It also shows resource shortnames, and whether objects are namespaced or cluster-scoped. The output has been tweaked to fit the book as well as show a mix of resources from different groups.

```
$ kubectl api-resources
NAME              SHORTNAMES   APIGROUP                NAMESPACED   KIND
namespaces        ns                                   false        Namespace
nodes             no                                   false        Node
pods              po                                   true         Pod
deployments       deploy       apps                    true         Deployment
replicasets       rs           apps                    true         ReplicaSet
statefulsets      sts          apps                    true         StatefulSet
cronjobs          cj           batch                   true         CronJob
jobs                           batch                   true         Job
ingresses         ing          networking.k8s.io       true         Ingress
networkpolicies   netpol       networking.k8s.io       true         NetworkPolicy
storageclasses    sc           storage.k8s.io          false        StorageClass
```

The next command shows which API versions are supported on your cluster. It doesn't list which resources belong to which APIs, but it's good for finding out whether you have things such as `alpha` APIs enabled or not. Notice how some API groups have multiple versions enabled such as beta and a stable.

```
$ kubectl api-versions
admissionregistration.k8s.io/v1
admissionregistration.k8s.io/v1beta1
apiextensions.k8s.io/v1
apiextensions.k8s.io/v1beta1
<Snip>
scheduling.k8s.io/v1beta1
storage.k8s.io/v1
storage.k8s.io/v1beta1
v1
```

The next one is a more complicated command that lists just the `kind` and `version` fields for resources supported on your cluster. The output is trimmed so that it gives you an idea of what you get. It doesn't work on Windows.

```
$ for kind in `kubectl api-resources | tail +2 | awk '{ print $1 }'`; \
 do kubectl explain $kind; done | grep -e "KIND:" -e "VERSION:"

KIND:     Namespace
VERSION:  v1
KIND:     Node
VERSION:  v1
<Snip>
KIND:     HorizontalPodAutoscaler
VERSION:  autoscaling/v1
KIND:     CronJob
VERSION:  batch/v1beta1
KIND:     Job
VERSION:  batch/v1
<Snip>
```

A quick word on accessing the API

While `kubectl` can be useful for getting API info, it's often better to explore the API more directly using one of the following options.

- API development tools
- Commands like `curl`, `wget`, and `Invoke-WebRequest`
- Web browser

The simplest way to do this, is to run a `kubectl proxy` command that exposes the API on your localhost adapter and handles all security and authentication.

The following command exposes the API on port 9000 on your localhost adapter. The proxy remains up until you kill the process (process ID 14781 in the example). If you run the command without the "&" on the end, you'll need to open a new terminal window to run commands.

You don't need to run this if you already have a kubectl proxy running.

```
$ kubectl proxy --port 9000 &
[1] 14781
Starting to serve on 127.0.0.1:9000
```

Kubectl is now proxying the API server on `localhost:9000`. Run some curl commands to explore the API.

The following two commands list all API versions available below the core API group, and the same for the named API groups.

```
$ curl http://localhost:9000/api
{
  "kind": "APIVersions",
  "versions": [
    "v1"
  ],
  "serverAddressByClientCIDRs": [
    {
      "clientCIDR": "0.0.0.0/0",
      "serverAddress": "172.21.0.4:6443"
    }
  ]
}

$ curl http://localhost:9000/apis
{
  "kind": "APIGroupList",
  "apiVersion": "v1",
  "groups": [
    <Snip>
    {
      "name": "apps",
      "versions": [
```

```
        {
          "groupVersion": "apps/v1",
          "version": "v1"
        }
      ],
      "preferredVersion": {
        "groupVersion": "apps/v1",
        "version": "v1"
      }
    },
    <Snip>
```

You can list specific object instances, or lists of objects on your cluster. The following shows a list of all Namespaces on a cluster.

```
$ curl http://localhost:9000/api/v1/namespaces
{
  "kind": "NamespaceList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "478774"
  },
  "items": [
    {
      "metadata": {
        "name": "kube-system",
        "uid": "a8c48564-d87d-41e0-af1d-1729c6849b52",
        "resourceVersion": "5",
        "creationTimestamp": "2021-01-17T17:44:59Z",
        "managedFields": [
          {
            "manager": "k3s",
            "operation": "Update",
            "apiVersion": "v1",
<Snip>
```

Feel free to poke around. You can put the same URI paths into a browser and API tools like Postman.

Leave the `kubectl proxy` process running as you'll use it again later in the chapter.

Alpha beta and stable

Kubernetes has a strict process for adding new resources to the API. They come in as *alpha*, progress through *beta*, and eventually reach *stable* status.

Resources in **alpha** are experimental and should be considered *hairy and scary*. Expect bugs, expect features to be dropped without warning, and expect lots of change as they graduate through beta to stable. A lot of clusters disable alpha APIs by default, and you should use them with extreme caution.

A resource that progresses through two alpha versions will go through the following APIs.

- /apis/some-api/v1alpa1/...
- /apis/some-api/v1alpa2/...

The phase after alpha is beta.

Resources in **beta** are considered "pre-release" and are starting to look and feel a lot like they will when they graduate to stable. Features that are part of beta objects will rarely be dropped when the resource graduates to stable. However, small changes should be expected. Most clusters enable beta APIs by default, and many people use beta objects in production environments. However, that's not a recommendation, you need to make those decisions yourself.

A resource that progresses through two beta versions will be served through the following APIs.

- /apis/some-api/vbeta1/...
- /apis/some-api/vbeta2/...

The final phase after beta is *stable*, sometimes referred to as *generally available (GA)*. Stable resources are considered production-ready, and Kubernetes has a strong long-term commitment to them.

Examples of paths to stable resources include the following.

- /apis/some-api/v1/...
- /apis/some-api/v2/...

Most stable resources are currently at `v1`, but occasionally some continue being developed and change so much that a `v2` is required. For an object to become `v2`, it progresses through alpha and beta again as follows.

- /apis/some-api/v2alpaX/... –>>
- /apis/some-api/v2betaX/... –>>
- /apis/some-api/v2

It's usually possible to deploy an object through a beta API, but continue to work with them through later API versions. For example, you can deploy an object via a `v1beta1` API, but update and manage it at a later date through the stable `v1` API.

Resource deprecation

As mentioned in the previous section, alpha and beta objects are subject to changes before graduating to stable. However, once an object is stable, Kubernetes has a strong commitment to maintaining long-term usability and support. Basically, once an object is stable (`v1, v2` etc.) it has a long life.

Kubernetes commits to support beta and stable objects as follows.

- Stable/GA: Objects are expected to be long-lived. When deprecated, stable objects will be supported for a further 12 months, or 3 releases, whichever's longest.
- Beta: Objects in beta have a 9 month window to either graduate to stable, or release a newer beta version. This is designed to avoid beta features going stale or staying in beta for too long like the Ingress resource did (Ingress remained in beta for more than 15 releases of Kubernetes).

Recent versions of Kubernetes return deprecation warning messages any time you use a deprecated resource. For example, deploying an Ingress from the old `extensions/v1beta1` API results in the following deprecation warning.

```
$ kubectl apply -f deprecate.yml
Warning: extensions/v1beta1 Ingress is deprecated in v1.14+, unavailable in v1.22+;
Use networking.k8s.io/v1 Ingress
```

Resources, objects, and primitives

You'll often see the terms *resources, objects,* and even *primitives* used interchangeably. And that's fine, most people do it and everyone understands what you mean.

However, technically speaking, the Kubernetes API is *resource-based.* This means everything in the API is a resource. It just so happens that most API resources, such as Pods, Services, Ingresses etc. are *objects*. However, some resources are lists, and an even smaller number are operations. However, most resources are objects, so we often use the terms "resource" and "object" to mean the same thing.

Resources can be either namespaced or cluster-scoped. Namespaced objects have to be deployed to a particular Namespace, whereas cluster-scoped objects can either be bound to multiple Namespaces, or exist entirely outside the realm of Namespaces. Node objects are an example of a cluster-scoped resource that exists entirely outside of Namespaces and cannot ever be deployed to one. On the flip side, ClusterRoles are an example of a cluster-scoped object that can be bound to specific Namespaces via ClusterRoleBindings.

Remember, `kubectl api-resources` lists all resources and whether they're namespaced or cluster-scoped.

Extending the API

Kubernetes runs a set of built-in controllers that read an object's `spec`, deploy and maintain it on the cluster, and keep its `status` section up-to-date. However, you can extend Kubernetes by adding your own resources and controllers.

A common example of 3rd-parties extending the Kubernetes API can be seen in the storage world where vendors expose advanced features, such as snapshot schedules, via custom resources in the Kubernetes API. In this model, storage is surfaced inside of Kubernetes via CSI drivers, Pods consume it via built-in Kubernetes resources such as StorageClasses and PersistentVoumeClaims, but advanced features such as snapshot scheduling is consumed via custom resources in the API.

The high-level pattern for extending the API involves two main things:

- Writing your custom *controller*
- Creating the custom *resource*

Kubernetes has a CustomResourceDefinition (CRD) object that lets you create new resources in the API that look, smell, and feel like native Kubernetes resources. This means you can create a custom resource, and then use `kubectl` to create and inspect it just like it's a native resource. They also get their own REST paths in the API.

The following YAML is from the `crd.yml` file in the `api` folder of the book's GitHub repo. It defines a new namespaced custom resource called "books" in the "nigelpoulton.com" named group that is served via the "v1" path.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: books.nigelpoulton.com
spec:
  group: nigelpoulton.com      <<-- Named API group
  scope: Cluster               <<-- Can be "Namespaced" or "Cluster"
  names:
    plural: books              <<-- All resources need a plural and singular name defined
    singular: book             <<-- Singular names are used on CLI and command outputs
    kind: Book                 <<-- kind property used in YAML files
    shortNames:
    - bk                       <<-- Short name used by kubectl
  versions:                    <<-- Resources can be served by multiple API versions
    - name: v1
      served: true             <<-- If set to false, "v1" will not be served
      storage: true            <<-- Store instances of the object as this version
      schema:                  <<-- This block defines the properties of your custom resource
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                <Snip>
```

Use the following command to make a local copy of the GitHub repo and the lab files. You'll need to have git installed for this to work.

```
$ git clone https://github.com/nigelpoulton/TheK8sBook.git
```

Change into the `api` directory.

```
$ cd TheK8sBook/api
```

If you're following along, deploy the custom resource with the following command. Be sure to run it from the `api` folder containing the `crd.yml` file.

```
$ kubectl apply -f crd.yml
customresourcedefinition.apiextensions.k8s.io/books.nigelpoulton.com created
```

Once deployed, the new resource exists in the API and you can deploy objects from it. This particular one will be served on the following REST path.

```
apis/nigelpoulton.com/v1/books/
```

Verify it exists in the API.

```
$ kubectl api-resources | grep books
NAME      SHORTNAMES     APIGROUP             NAMESPACED     KIND
books     bk             nigelpoulton.com     false          Book

$ kubectl explain book
KIND:     Book
VERSION:  nigelpoulton.com/v1
DESCRIPTION:
     <empty>
FIELDS:
     <Snip>
```

The following YAML is from the `tkb.yml` file in the `api` folder of the book's GitHub repo. It defines a new `Book` object called "tkb". Notice how the fields in the `spec` section match the names and types defined in the custom resource YAML definition (`crd.yml`).

```
apiVersion: nigelpoulton.com/v1
kind: Book
metadata:
  name: tkb
spec:
  bookTitle: "The Kubernetes Book"
  topic: Kubernetes
  edition: 2
```

Deploy it with the following command.

```
$ kubectl apply -f tkb.yml
book.nigelpoulton.com/tkb created
```

You can now list and describe it with the usual commands. The following GET command uses the resource's "bk" shortname.

```
$ kubectl get bk
NAME   TITLE                 EDITION
tkb    The Kubernetes Book   2
```

Finally, you can use tools like `curl` to query the new API group and resource. The simplest way to do this is to run a kubectl proxy.

The following commands start a kubectl proxy and list all resources under the new `nigelpoulton.com` named group. You may already have a `kubectl proxy` process running.

```
$ kubectl proxy --port 9000 &
[1] 14784
Starting to serve on 127.0.0.1:9000

$ curl http://localhost:9000/apis/nigelpoulton.com/v1/
{
  "kind": "APIResourceList",
  "apiVersion": "v1",
  "groupVersion": "nigelpoulton.com/v1",
  "resources": [
    {
      "name": "books",
      "singularName": "book",
      "namespaced": false,
      "kind": "Book",
      "verbs": [
        "delete",
        "deletecollection",
        "get",
        "list",
        "patch",
        "create",
        "update",
        "watch"
      ],
      "shortNames": [
        "bk"
      ],
      "storageVersionHash": "rQ0xv5vJ3/s="
    }
  ]
}
```

This is all good an interesting. But a custom resource doesn't do anything until you create a custom controller to go with it. Doing this is beyond the scope of this chapter, but you've learned a lot about the Kubernetes API and how it works.

Chapter summary

Now that you've read the chapter, all of the following should make sense. But don't worry if some bits are still vague or confusing. APIs can be hard to understand, and the Kubernetes API is large and complex. Anyway, here goes…

Kubernetes is API centric, and the API is exposed internally and externally via the API server.

The API server runs as a control plane service, and all internal and external clients interact with each other and the API, via the API server. This means your control plane needs to be highly available and high performance. If it's not, you risk slow API response times or entirely losing access to the API. Also, all requests to the API server are authenticated, authorized, and protected by TLS.

The API itself is a modern resource-based RESTful API that accepts CRUD-style operations via uniform HTTP methods such as POST, GET, PUT, PATCH, and DELETE. It's divided into named groups for convenience and extensibility. Older resources created in the early days of Kubernetes exist in the original core group which is accessed via the `/api/v1` REST path. All newer objects go into named groups. For example, newer network resources are defined in the `networking.k8s.io` sub-group available at the `/apis/networking.k8s.io/v1/` REST path.

Resources in the Kubernetes API are generally objects. However, they can also be lists or operations. The vast majority are objects, so we sometimes use the terms "resources" and "objects" to mean the same thing. It's common to refer to their API definitions as resources, or resource definitions, whereas running instances on a cluster are often referred to as objects. For example, *the Pod "resource" exists in the core API group, and there are 5 Pod "objects" running in the default Namespace.*

All new resources come into the API as alpha, progress through beta, and eventually graduate to stable. Alpha resources are subject to change and disabled in many clusters due to their unstable nature. Beta resources are more stable and consist of features expected to be carried through to the stable version. Most clusters enable beta resources by default, but you should be cautious using them in production. Stable resources are considered production-grade and Kubernetes has a strong commitment to them that is backed by a clear deprecation policy that guarantees they'll be supported for at least 12 months, or three versions, after the deprecation announcement.

Finally, the Kubernetes API is becoming the de facto cloud API with many 3rd-party technologies extending it so they can be exposed through it. Kubernetes makes it easy to extend the API with your own custom resources through CustomResourceDefinitions that make your custom resources look like native Kubernetes resources.

OK, hopefully that made sense. But don't worry if you're still a bit unsure about some of the points. I highly recommend you play around with as many of the examples as possible. Also consider reading the chapter again in a day or so, as it often takes time and more than one reading of something before you grasp it.

Finally, if you liked this chapter, or any other chapter in the book, jump over to Amazon and show the book some love with a quick review. The cloud-native gods will smile on you ;-)